

Práctica 1: PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Edward Andres Pacheco Jurado - 2222236
Angely Yulitza Vásquez Lozada - 2222240
Daniel Eduardo Suarez Jaimes - 2222228

https://github.com/EDWARDPJ26/2026_2_CommII_A1_GA1D

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—This work presents the implementation and evaluation of basic digital signal processing techniques using GNU Radio as a Software Defined Radio (SDR) environment. Different processing blocks are used to analyze the behavior of discrete-time signals, including signal accumulation, differentiation, and statistical analysis. The experiments involve the generation and visualization of different types of signals, allowing the response of each processing stage to be observed. Through these implementations, the practical behavior of the algorithms is compared with their theoretical principles, providing a better understanding of digital signal processing and its application in SDR systems.

Index Terms—GNU Radio, Software Defined Radio, digital signal processing, accumulator, differentiator, statistical analysis, discrete-time signals.

I Introducción

La Radio Definida por Software (SDR) es un enfoque importante en los sistemas de comunicación modernos, ya que permite implementar y modificar diferentes operaciones de procesamiento de señales mediante software. Esto proporciona un entorno flexible para estudiar cómo se generan, procesan y analizan las señales sin depender únicamente de implementaciones basadas en hardware fijo.

GNU Radio es una plataforma de software que permite diseñar sistemas de procesamiento de señales mediante bloques interconectados. En este entorno es posible trabajar con señales en tiempo discreto y observar su comportamiento en diferentes etapas de una cadena de procesamiento. Esto lo convierte en una herramienta útil para comprender de manera práctica conceptos relacionados con el procesamiento digital de señales y las comunicaciones.

En esta práctica se utiliza GNU Radio para implementar y probar operaciones básicas de procesamiento,

como la acumulación y la diferenciación de señales. Se aplican diferentes señales de entrada a estos bloques con el fin de observar sus respuestas y compararlas con el comportamiento teórico esperado. Además, el análisis estadístico hace parte de la práctica para obtener información sobre las características de las señales procesadas. Estas actividades permiten reforzar de manera práctica los conceptos estudiados en comunicaciones digitales y comprender cómo pueden implementarse algoritmos básicos de procesamiento de señales en un entorno de SDR. [2]

II Metodología

La práctica se desarrolló en GNU Radio utilizando una estructura basada en bloques para la generación, procesamiento y visualización de señales en tiempo discreto. En los experimentos se utilizó una frecuencia de muestreo de 32 kHz. La metodología se dividió en la implementación del acumulador, el diferenciador y la etapa de análisis estadístico.

II-A Bloque Acumulador

El bloque acumulador se implementó mediante un Embedded Python Block y se conectó entre un Vector Source y un QT GUI Time Sink. Se generó una señal cuadrada periódica mediante el Vector Source y se procesó con una frecuencia de muestreo de 32 kHz. Utilizando el libro guía de comunicaciones plus se diseñó la estructura del código de Python y el diagrama de bloques.[1, p. 19]

El acumulador calcula la suma progresiva de las muestras de entrada mediante:

$$y[n] = \sum_{k=0}^n x[k] \quad (1)$$

donde $x[k]$ representa las muestras de entrada y $y[n]$ corresponde a la salida acumulada. Esta operación permite observar el comportamiento de un integrador en tiempo discreto.

Para la prueba inicial se utilizó una señal cuadrada con niveles de amplitud de $+1$ y -1 . La señal de entrada y la salida acumulada se visualizaron simultáneamente mediante el QT GUI Time Sink.

II-B Bloque Diferenciador

El diferenciador se implementó mediante un Embedded Python Block conectado a un Vector Source, un Throttle y un QT GUI Time Sink. Se utilizaron diferentes señales en tiempo discreto para evaluar la respuesta del bloque, manteniendo una frecuencia de muestreo de 32 kHz. Utilizando el libro guía de comunicaciones plus se diseñó la estructura del código de Python y el diagrama de bloques. [1, p. 19]

La operación realizada por el diferenciador está dada por:

$$y[n] = x[n] - x[n-1] \quad (2)$$

donde $x[n]$ corresponde a la muestra actual de entrada y $x[n-1]$ a la muestra anterior. Por lo tanto, la salida representa el cambio entre dos muestras consecutivas.

Para las pruebas se utilizaron dos vectores de entrada. El primero correspondió al vector constante (5,5), mientras que el segundo correspondió al vector (1,2,3,4,5,6). Las señales de entrada y salida fueron visualizadas simultáneamente mediante el QT GUI Time Sink para facilitar su comparación.

II-C Análisis Estadístico

Como parte de la práctica completa, también se considera una etapa de análisis estadístico para caracterizar las señales generadas. Entre los principales parámetros se encuentran la media, el valor RMS, la desviación estándar y la potencia promedio.

La media de una señal discreta se calcula mediante:

$$\bar{x} = \frac{1}{N} \sum_{n=0}^{N-1} x[n] \quad (3)$$

El valor RMS se obtiene mediante:

$$x_{\text{RMS}} = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} x^2[n]} \quad (4)$$

La media cuadrática se puede calcular cómo:

$$x_{\text{RMS}} = \frac{1}{N} \sum_{n=0}^{N-1} x^2[n] \quad (5)$$

La desviación estándar se puede calcular cómo:

$$\sigma = \sqrt{\frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \mu)^2} \quad (6)$$

Por otra parte, la potencia promedio puede calcularse como:

$$P = \frac{1}{N} \sum_{n=0}^{N-1} x^2[n] \quad (7)$$

donde N representa el número de muestras del vector analizado. Esta etapa permite caracterizar estadísticamente las señales y estudiar sus propiedades bajo diferentes condiciones de procesamiento.

II-D Bloque Estadístico

El diferenciador se implementó mediante un Embedded Python Block conectado a un Vector Source, un Throttle y a varios QT GUI Time Sink. Se aplicó constantemente una señal discreta, manteniendo una frecuencia de muestreo de 32 kHz. Utilizando el libro guía de comunicaciones plus se diseñó la estructura del código de Python y el diagrama de bloques. [1, pp. 20–21]

Las operaciones del bloque estadístico son las definidas en el análisis estadístico siendo el valor de la media (4), el valor RMS (5), el valor de la media cuadrática (6), el valor de la desviación estándar (7) y el valor de la potencia (8).

III RESULTADOS Y ANÁLISIS DE DATOS

III-A Bloque Acumulador

Para evaluar el bloque acumulador se utilizó una señal cuadrada con amplitudes de *image*, una frecuencia de muestreo de 32 kHz y un vector de ocho muestras. La señal de entrada y la señal obtenida a la salida del acumulador se visualizaron mediante los bloques QT GUI Time Sink, como se observa en la Figura 1.

En la gráfica obtenida en la Figura 2 se observa que la señal cuadrada de entrada es transformada en una señal de tipo triangular o rampa. Esto ocurre debido a que el bloque realiza la suma acumulativa de las muestras de entrada. Mientras la señal mantiene un valor positivo, la salida aumenta progresivamente; cuando la entrada toma un valor negativo, la salida comienza a disminuir.

Este comportamiento confirma experimentalmente la función del acumulador como un integrador en tiempo

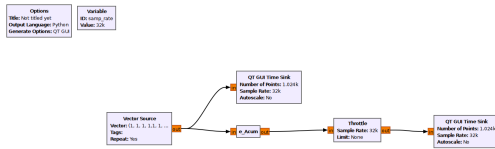


Figura 1: Diagrama de bloques del acumulador

discreto, ya que su salida depende tanto de la muestra actual como de los valores acumulados anteriormente. Además, se observa que la salida conserva información de las muestras previas, característica propia de este tipo de procesamiento.

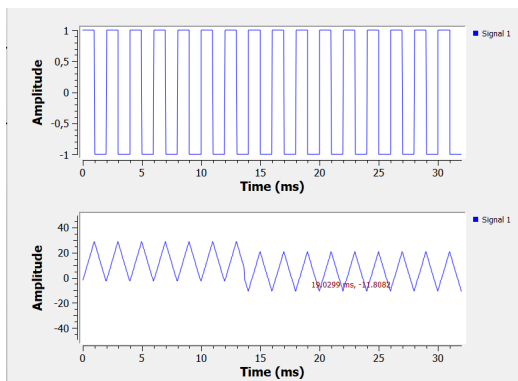


Figura 2: Señal de entrada y salida del bloque acumulador.

III-B Bloque Diferenciador

Para comprobar el funcionamiento del bloque diferenciador se realizaron dos pruebas utilizando diferentes vectores de entrada, manteniendo una frecuencia de muestreo de 32 kHz, el diagrama de bloque se GNUradio se encuentra en la Figura 3.

En la primera prueba se utilizó el vector como una constante. Como las dos muestras tienen el mismo valor, la diferencia entre ellas es cero:

$$y[n] = 5 - 5 = 0 \quad (8)$$

Por esta razón, en la gráfica de la Figura 3, la salida permanece en cero durante todo el intervalo observado. Esto demuestra que el diferenciador no genera una respuesta cuando la señal de entrada permanece constante.

En la segunda prueba se utilizó el vector como rampa. Debido a que cada muestra aumenta en una unidad con respecto a la anterior, el bloque produce una salida aproximadamente constante de valor *image*:

$$2 - 1 = 1, \quad 3 - 2 = 1, \quad 4 - 3 = 1 \quad (9)$$

y de manera similar para las muestras restantes. En la gráfica se observa este comportamiento mediante una señal de salida. Al producirse el cambio brusco asociado con la repetición de la secuencia, aparece un pico negativo, debido a la diferencia entre el nuevo valor y la muestra anterior.

Los resultados obtenidos permiten comprobar que el bloque diferenciador responde directamente a los cambios entre muestras consecutivas, generando valores mayores cuando se presentan variaciones más pronunciadas en la señal.

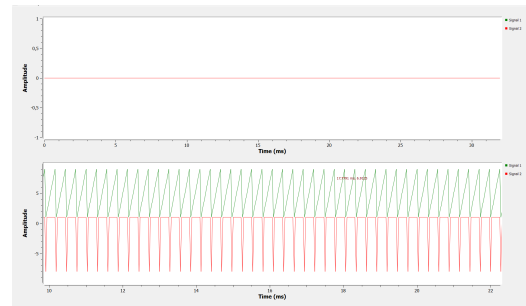


Figura 3: Respuesta del bloque diferenciador para los vectores de entrada utilizados.

III-C Bloque Estadístico

Para comprobar el funcionamiento del bloque estadístico se aplicó un vector de prueba a la entrada, manteniendo una frecuencia de muestreo de 32 kHz. El diagrama de bloque se GNUradio se encuentra en la Figura 4.

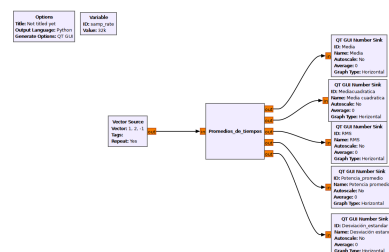


Figura 4: Diagrama del bloque estadístico.

En la prueba se utilizó el vector:

$$[1, 2, -1] \quad (10)$$

Este vector esta se esta aplicando en la entrada constantemente lo que indica que los valores del bloque

estadístico van estar variando durante el tiempo. En la Figura 5 se muestra la simulación del bloque ante la entrada mencionada, de este modo se observa el resultado de cada valor estadístico definido.

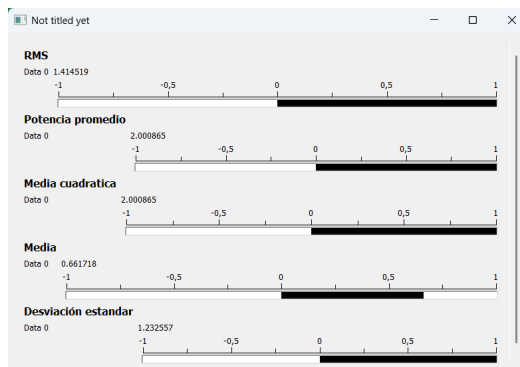


Figura 5: Respuesta del bloque estadístico para el vector de entrada.

IV Conclusiones

- Se comprobó experimentalmente el funcionamiento del **bloque acumulador**, observando que una señal cuadrada de entrada genera una señal acumulativa con comportamiento similar al de un integrador en tiempo discreto.
- El **bloque diferenciador** permitió identificar los cambios presentes entre muestras consecutivas. Para una señal constante, la salida fue cero, mientras que para una secuencia con incrementos uniformes se obtuvo una salida aproximadamente constante.
- El **bloque estadístico** es importante porque permite caracterizar una señal a partir de diferentes medidas estadísticas, facilitando la identificación de variaciones y comportamientos relevantes a lo largo del tiempo. Esto resulta especialmente útil cuando el vector de entrada contiene miles o millones de muestras, ya que permite resumir y analizar grandes cantidades de información de manera más sencilla. Además, el análisis estadístico puede ayudar a detectar anomalías o variaciones en la señal producidas por procesos de transmisión, ruido o perturbaciones presentes en los dispositivos y sistemas de comunicación.
- Las pruebas realizadas permitieron relacionar directamente el comportamiento observado en GNU Radio con las operaciones matemáticas de **acumulación y diferenciación en tiempo discreto**, facilitando la comprensión de estos conceptos mediante su implementación práctica.
- Finalmente, la práctica permitió reforzar los con-

ceptos de procesamiento digital de señales y comprender cómo operaciones matemáticas básicas pueden ser implementadas y observadas directamente en un entorno de **Radio Definida por Software (SDR)**.

Referencias

- [1] Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones, *Libro de Comunicaciones Plus Volumen I*. Bucaramanga, Colombia: Universidad Industrial de Santander, 2020, material de apoyo académico.
- [2] GNU Radio Project, "GNU Radio Wiki: Main Page," [En línea]. Disponible en: https://wiki.gnuradio.org/index.php/Main_Page. [Accedido: 31-ago-2026].